

What runs where

Every piece of technology in this system, what it actually does in plain words, and the exact file it lives in. Written for someone who is not an engineer.

The one-line version

The system reads records the force already keeps, looks for people whose pattern has changed, and then argues itself out of naming almost all of them.

Three of two hundred and forty reach a human on a typical night. Nearly everything below exists to make that number small and to make each of the three defensible.

Nine stops, and the tool at each one

These are numbered because they genuinely happen in this order. Each stop says what happens in plain words, then names the tool and the file.

1 Collect the records

Seven separate readers each fetch one kind of record — leave, duty roster, deployments, transfers, training, workload, and voluntary self-assessments. Think of seven clerks, each sent to one filing cabinet.

Python + a file reader `ingest/connectors/hrms.py`

Why a file, not a live connection: the HR system writes a nightly export to a shared folder and we read it. Nothing dials out, no password is stored on the analytics side, and a records server being down cannot stop a welfare run.

2 Replace every name with a ticket number

Before anything is analysed, each service number is swapped for a scrambled code. From here on, the system is working with 7c4e2b91af... and not with a person. The scrambling is one-way — you cannot work backwards from the code.

HMAC-SHA256 (built into Python) `ingest/pseudonymise.py`

Why it matters: the consent check runs *after* this step, so even the part of the system that decides who agreed to be monitored never sees a name.

3 Work out what normal looks like — for that person

For each person the system builds their own history over 7, 30, 90 and 180 days, and separately the history of their whole unit. A jawan who has always worked long hours has not changed. One whose hours doubled last month has.

Plain Python + NumPy `analytics/features/store.py`

Why two baselines: comparing everyone to one fixed standard would flag every person in a hard posting and nobody in an easy one — the opposite of welfare.

4 Spot who has moved away from their own normal

A statistical check per person per record type. Out of 240 people, roughly 128 show a change somewhere. That is not 128 problems — it is 128 things worth looking at properly.

NumPy `analytics/features/deviation.py`

Numbers only. No machine learning has run yet at this point.

5 The AI has its say

Now the machine-learning model runs. It has learned patterns that often precede difficulty and gives each person a score. A second model estimates *by when* — an even chance within thirty days, say — which is what turns a score into something you can put in a diary.

scikit-learn – gradient boosting `analytics/models/tabular.py`

Why this one: the submitted proposal named LightGBM, which is faster. It is still supported and it is not the default, because it needs an extra system library that a copied folder will not have. Same family of algorithm, same explanations, one fewer thing to go wrong on a strange machine.

SHAP `analytics/attribution.py`

The interpreter. A score on its own is useless to an officer. SHAP takes the model apart and says which facts drove it, and the system translates that into a sentence a person can repeat aloud: `not roster_var_30d = 0.41` but “their shift pattern has been irregular for a month”.

6 Three reviewers argue about it

One argues the case for concern. One deliberately looks for innocent explanations — is the whole unit deployed, is this planned leave, is it a training cycle. One checks what was tried for this person before. They run at the same time and disagree on the record.

Plain Python, rule-based `analytics/reviewers/`

No chatbot anywhere. These are rules a person can read, not a language model. There is no ChatGPT-style component in this system and no request ever leaves the building.

7 Four checks decide — and this is the important stop

Four sums, each answering one question. Is there enough evidence? Do the signals agree with each other? Has it lasted? Is there actually something we can offer? **All four must pass.** If any one fails, no name is produced — and the system states what would have changed its answer.

Plain Python arithmetic `core/gates.py`

Deliberately boring, and deliberately not AI. The model suggests; this decides. A test scans this file and fails the build if anyone puts a database call, a clock, a random number or a model into it. It is the one part of the system a governance board can read line by line and check with a calculator.

8 Write it all down, in a book that cannot be edited

Every decision, every access, every disclosure goes into a log where each entry is sealed with a code derived from the entry before it. Tear a page out and every page after it stops matching — like a numbered receipt book where the stubs must add up.

SHA-256 hash chain `disclosure/audit.py`

The rule that makes it real: the log entry is written *before* a name is looked up, and if the write fails the disclosure is cancelled. There is no sequence of events in which somebody sees a name without a record of it.

9 Show it to the right person, and only them

Four different screens for four different jobs. The welfare officer sees names. The commanding officer never does. The auditor sees every decision but no identities. The individual sees only themselves.

FastAPI `api/app.py`

The counter. It receives every request and checks who is asking before answering. The rules live here rather than in the screens, so a bug in a screen cannot leak a name — the answer never contained one.

Next.js + React + TypeScript `web/`

The screens. Also installable on a phone from the force's own network, which is the “mobile app” the brief asks for — no app store involved, so the force controls it end to end.

THE PART THE PROBLEM STATEMENT ASKS ABOUT MOST

How the data is protected

WHAT	TOOL	IN PLAIN WORDS
Names → codes	HMAC-SHA256	A one-way scramble. The code cannot be turned back into a service number without a secret held separately.
Locked storage	AES-256-GCM	The one table that maps a code back to a person is encrypted. Someone who steals the database file gets codes and gibberish. Each record is also tied to its own code, so a thief cannot move one person's locked record onto another person's row.
Where the key lives	keyfile / HSM	Never beside the data, never inside the project folder, and never readable by other users. The system refuses to start rather than run with a key it considers unsafe.
Small groups hidden	k-anonymity	A figure covering fewer than five people is not shown at all — it is blanked, not rounded. Rounding still leaks.
Deliberate fuzzing	differential privacy	Group figures get a little noise added, so no individual can be worked out by comparing two views. Refreshing the page gives the <i>same</i> answer back, because otherwise averaging several refreshes would cancel the noise out.
Training without pooling	federated averaging	Units can improve the model by sharing what it learned rather than sharing their records. The records never leave the unit.
Deleting means deleting	secure delete + vacuum	When a record is removed, the actual bytes are overwritten and the file rebuilt. A row merely “marked deleted” is still sitting there for anyone who looks.

Things the proposal named that we did differently

Nine of the technologies in the submitted proposal are running as written. Five were swapped, and every swap has the same reason: **the whole system had to survive being copied onto a pendrive and emailed**, then run on a machine with no internet.

PROPOSED	WHAT WE RUN	WHY, IN ONE LINE
LightGBM	scikit-learn (LightGBM optional)	LightGBM needs an extra system library that a copied folder will not have. It installs fine and then fails to start.
Apache Airflow	Our own nine-stage runner	Airflow is for scheduling hundreds of jobs across a cluster. This is one job, once a night.
Redis	In-memory cache	Same thing, no extra server. And here it is a privacy control, not a speed-up.
Flutter app	Installable web app	No app store, so the force hosts and controls it — and there is one consent screen to maintain instead of three.
Temporal Transformer	Survival model + roster features	The deep-learning library is about 2 GB. It cannot be emailed.
Kubernetes	One server	Measured at 7.7 milliseconds per person. Ten lakh personnel finishes in about two hours on a single machine.

Nothing was dropped

Each swapped-out technology is either available as an optional add-on or sits behind a connector designed for it, so a data centre that wants the original choice can have it without rewriting anything.

What the model was trained on

There is no real dataset, and there is no dataset file.

The data is **generated by the system itself** from a fixed starting number (seed 26186) each time the model is trained. Same seed, same 1,680 synthetic personnel, every time. Nothing is downloaded and no file sits on disk waiting to be read.

The model's own record card states this — "synthetic": `true` — and it is stamped there so that no accuracy figure from it can be quoted as validation by accident.

And the part that makes the accuracy figure meaningless

The generator decides how strained each invented person is. The training answer key is then *that same number* — anyone above 0.55 counts as a positive case. So the model is being asked to rediscover a fact the generator already knew. It scores well (AUROC 0.89) because the exercise is circular, not because it can predict distress in a real force.

What the synthetic data does prove is that the machinery works end to end: the pipeline runs, the gates compute, the refusals happen for the stated reasons, and the privacy controls hold. **What it cannot prove** is that the model is any good. That needs a real cohort, an ethics approval and a data-sharing agreement.

One figure in the card is worth pointing at deliberately: on the *deployed* subgroup the model scores 0.50 — no better than a coin toss. That is the exact case the four checks exist to catch, and it is published rather than hidden.

QUESTION	ANSWER
Where is the dataset file?	There isn't one. It is generated in memory when training runs.
What generates it?	<code>src/samvedna/ingest/generator.py</code>
Where is the answer key set?	<code>tools/train_tabular.py</code> , line 88
Where is the provenance recorded?	<code>artefacts/models/tabular_card.json</code>
How do I reproduce it exactly?	<code>uv run --extra ml python tools/train_tabular.py --units 12 --strength 140 --seed 26186</code>

QUESTION	ANSWER
How big is it?	1,680 personnel — 1,176 for training, 504 for testing. 3.2% positive.

TO SEE ANY OF THIS YOURSELF

Running it

COMMAND	WHAT IT DOES
<code>./run.sh</code>	Starts both servers and prints the address
<code>./run.sh --stop</code>	Stops them
<code>./run.sh --status</code>	Shows what is running
<code>./check.sh</code>	Runs every test and every safety check
<code>uv run samvedna cohorts</code>	The decision engine on its own, no screens

The two addresses

`http://localhost:3100/demo` — the four screens.

`http://localhost:8090/api-console` — the backend, showing every request the system can answer and which role is allowed to make it. Built without any external dependency so it still works with no internet, which the standard tool for this does not.

Every figure here was read from the running system.

Full engineering detail: `docs/TECH_STACK.md` · problem-statement audit: `docs/PS_COMPLIANCE.md`

Welfare support, never disciplinary action. Outputs are barred in software from ACR, posting, promotion and disciplinary processes.